

AI Shopping Assistant - Developer Setup Guide

This guide provides step-by-step instructions for setting up and running the AI Shopping Assistant project locally.

Prerequisites

Before you begin, ensure you have the following installed:

- **Node.js** v18 or higher
- **Python** 3.11 or higher
- **PostgreSQL** database (or use Replit's built-in database)
- **npm** or **yarn** package manager

Project Structure

```
├── backend/                                # Python FastAPI backend
│   ├── agents/                            # AI agent implementations
│   │   ├── orchestrator.py               # Main conversation orchestrator
│   │   ├── clarifier.py                  # Intent clarification agent
│   │   ├── product_recommender.py
│   │   └── base.py                       # Base agent class & LLM config
│   ├── database/                         # Database connection & models
│   ├── models/                           # Pydantic schemas
│   └── main.py                           # FastAPI application
├── client/                                # React frontend
│   ├── src/
│   │   ├── components/                  # React components
│   │   ├── styles/                     # CSS files
│   │   ├── pages/                       # Page components
│   │   └── App.tsx                      # Main app component
│   └── index.html
├── server/                               # Express.js proxy server
│   ├── index.ts                         # Server entry point
│   └── routes.ts                        # API routes
├── shared/                               # Shared types & schemas
│   └── schema.ts                       # Drizzle database schema
├── docs/                                 # Documentation
└── package.json                         # Node.js dependencies
```

```
|— pyproject.toml      # Python dependencies
|— vite.config.ts      # Vite configuration
```

Environment Variables

Create or update the following environment variables:

Required Variables

Variable	Description	Example
<code>DATABASE_URL</code>	PostgreSQL connection string	<code>postgresql://user:pass@host:5432/db</code>
<code>AZURE_OPENAI_API_KEY</code>	Azure OpenAI API key	<code>your-api-key</code>
<code>AZURE_OPENAI_ENDPOINT</code>	Azure OpenAI endpoint URL	<code>https://your-resource.openai.azure.com/</code>
<code>AZURE_OPENAI_DEPLOYMENT</code>	Deployment name (defaults to gpt-4o-mini)	<code>gpt-4o-mini</code>

Note: The system automatically handles variable order - if endpoint and API key are swapped, the backend will correct it.

Optional Variables

Variable	Description	Default
<code>OPENWEATHER_API_KEY</code>	OpenWeather API key for weather data	Uses Open-Meteo (free)
<code>NODE_ENV</code>	Environment mode	<code>development</code>

Setting Environment Variables

On Replit: 1. Go to the Secrets tab (lock icon) 2. Add each variable with its value

Locally: Create a `.env` file in the project root:

```
DATABASE_URL=postgresql://user:password@localhost:5432/shopping_assistant
AZURE_OPENAI_API_KEY=your-key-here
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
AZURE_OPENAI_DEPLOYMENT=gpt-4o-mini
```

Installation

1. Clone the Repository

```
git clone <repository-url>
cd ai-shopping-assistant
```

2. Install Node.js Dependencies

```
npm install
```

3. Install Python Dependencies

Using pip:

```
pip install -r requirements-local.txt
```

Or using uv (faster):

```
uv sync
```

4. Database Setup

The application uses PostgreSQL with Drizzle ORM for the frontend proxy and SQLAlchemy for the Python backend.

Create the database:

```
# Push schema to database
npm run db:push
```

Seed the database: The database is automatically seeded with sample products and customers on first startup.

Running the Application

Development Mode

Start the full application (frontend + backend):

```
npm run dev
```

This command: 1. Starts the Vite dev server for the frontend 2. Starts the Express proxy server on port 5000 3. Starts the Python FastAPI backend on port 8000

Access the application: - Frontend: `http://localhost:5000` - Backend API: `http://localhost:8000` - API Docs (Swagger): `http://localhost:8000/docs`

Running Components Separately

Frontend only:

```
npm run dev:client
```

Backend only:

```
cd backend  
python -m uvicorn main:app --reload --port 8000
```

Production Build

Build the application for production:

```
npm run build
```

Start the production server:

```
npm start
```

Database Schema

Key Tables

customers - `id` : Primary key - `customer_id` : Unique customer identifier (e.g., "CUST-0000001") - `first_name`, `last_name`, `email` - `password` : Plain text (for demo purposes) - `preferences`, `style_profile`, `size_info` : JSON fields

products - `id` : Primary key - `name`, `description`, `category`, `subcategory` - `price`, `brand`, `gender` - `sizes_available`, `colors`, `tags` : Array fields - `image_url`, `in_stock`, `rating`

conversations - `id` : Primary key - `customer_id` : Foreign key to customers - `messages` : JSON array of chat messages - `context` : JSON object with conversation context

API Architecture

Request Flow

```
Browser → Express Proxy (5000) → FastAPI Backend (8000)
      ↓
    PostgreSQL Database
```

Multi-Agent System

The backend uses a LangGraph-based multi-agent architecture:

1. **Orchestrator**: Coordinates all agents and manages conversation flow
2. **Clarifier Agent**: Captures user intent through conversational clarification
3. **Customer 360 Agent**: Fetches customer profile and preferences
4. **Context Aggregator**: Combines intent, customer, and environmental data
5. **Product Recommender**: RAG-based product recommendations

Development Tips

Type Checking

```
npm run check
```

Viewing Logs

Backend logs appear in the terminal where you ran `npm run dev`.

Testing API Endpoints

Use the Swagger UI at `http://localhost:8000/docs` or curl:

```
# Test chat endpoint
curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "I want to travel to Paris", "user_id": 1}'
```

Debugging

- Frontend: Use browser DevTools (F12)
- Backend: Logs print to console with `[DEBUG]` prefix
- Database: Use `npm run db:push` to sync schema changes

Common Issues

Database Connection Failed

- Verify `DATABASE_URL` is correct
- Check PostgreSQL is running
- Ensure database exists

Azure OpenAI Errors

- Verify API key and endpoint are correct
- Check deployment name matches your Azure configuration
- Ensure you have quota available

Port Already in Use

- Kill processes on ports 5000 and 8000
- `lsof -i :5000` and `kill -9 <PID>`

Python Import Errors

- Ensure you're in the project root directory

- Verify Python dependencies are installed
- Check Python version is 3.11+

Deployment

Replit Deployment

1. Click the "Deploy" button in Replit
2. Configure environment variables in Secrets
3. The app will be available at `your-app.replit.app`

Manual Deployment

1. Build the application: `npm run build`
2. Set production environment variables
3. Run: `npm start`
4. Configure reverse proxy (nginx) if needed

Contributing

1. Create a feature branch
2. Make changes
3. Run type checking: `npm run check`
4. Test thoroughly
5. Submit a pull request

Support

For issues or questions: - Check existing documentation - Review error logs - Open an issue on the repository